

Δεύτερη Φάση Προγραμματιστικής Εργασίας στην Σημασιολογική Ανάλυση

Υλοποίηση συστήματος αναζήτησης μέσω διανυσματικών αναπαραστάσεων transformers και FAISS indexing

Η παρούσα φάση της εργασίας υλοποιεί ένα σύγχρονο σύστημα σημασιολογικής αναζήτησης που συνδυάζει μετασχηματιστές (*transformers*) για τη δημιουργία πυκνών διανυσματικών αναπαραστάσεων κειμένων και τη βιβλιοθήκη *FAISS* για αποδοτική αναζήτηση στον πολυδιάστατο χώρο με σκοπό τον εντοπισμό κειμένων που σχετίζονται εννοιολογικά με το ερώτημα, ακόμη και όταν δεν μοιράζονται κοινές λέξεις. Επίσης, όπως και στην πρώτη φάση, ασχοληθήκαμε και με το εργαλείο αξιολόγησης *trec_eval*, το οποίο μας έδωσε *map* και *precision* για τις απαντήσεις μας και έτσι μπορέσαμε να τις αξιολογήσουμε. Συγκεκριμένα βήμα-βήμα:

1. Προεπεξεργαστήκαμε τη συλλογή κειμένων IR2025 (αρχείο [documents.csv](#)) προκειμένου να είναι σε κατάλληλη μορφή για να δοθεί ως είσοδος στο μοντέλο *transformers*. Συγκεκριμένα, αφαιρέσαμε τα *whitespace* και κανονικοποιήσαμε τα κείμενα.

```
11 #1. Preprocessing the texts
12 df = pd.read_csv("documents.csv")
13 df["Text"] = df["Text"].fillna("")
14
15 def preprocess_text(text: str):
16     text = text.strip()
17     text = regex.sub(r"\s+", " ", text)
18     text = regex.sub(r"\n+", " ", text)
19     return text
20
21 df["preprocessed_text"] = df["Text"].apply(preprocess_text)
22 df = df[df["preprocessed_text"].str.len() > 0]
23
24 documents = df["preprocessed_text"].tolist()
25
```

2. Στη συνέχεια, χρησιμοποιήσαμε το μοντέλο *all-MiniLM-L6-v2* για να παράγουμε τα πυκνά διανύσματα-ενθέσεις, τα οποία και αποθηκεύσαμε σε ένα αρχείο [document_embeddings.npy](#) την πρώτη φορά και έκτοτε κάθε φορά που τρέχαμε τον υπόλοιπο κώδικα, λαμβάναμε τις ενθέσεις απευθείας από το αρχείο για εξοικονόμηση χρόνου.

```

26 #2. Text to dense-vectors/embeddings conversion with SentenceTransformer
27 model = SentenceTransformer("all-MiniLM-L6-v2") # Dimension: 384
28
29 # Uncomment the following lines and comment line 39 to compute embeddings from scratch
30 '''
31 embeddings = model.encode(
32     documents,
33     batch_size=64,
34     show_progress_bar=True
35 )
36
37 np.save("document_embeddings.npy", embeddings)
38 '''
39 # Loading pre-computed embeddings
40 embeddings = np.load("document_embeddings.npy")
41

```

3. Προχωρώντας στο τρίτο ζητούμενο όπου και δημιουργήσαμε το ευρετήριο *IndexFlatIP* με συνημιτονοειδή ομοιότητα.

```

43 #3. Building the FAISS index with IndexFlatL2
44 dimension = embeddings.shape[1]
45 index = faiss.IndexFlatIP(dimension)
46 faiss.normalize_L2(embeddings) # Normalize for cosine similarity
47 index.add(embeddings)

```

4. Για το τέταρτο ζητούμενο ανακτήσαμε για κάθε ερώτημα από το αρχείο *queries.csv* τα *k* πιο κοντινά έγγραφα (*k=20,30,50*). Για να γίνει αυτό, α) δημιουργήσαμε *embeddings* για κάθε ερώτημα, β) αναζητήσαμε τους *k* πλησιέστερους γείτονες στο *FAISS* και γ) ταξινομήσαμε τα αποτελέσματα κατά φθίνουσα ομοιότητα. Σε αυτό το σημείο αξίζει να αναφερθεί πως αξιοποιήθηκε *batch processing* για όλα τα ερωτήματα ταυτόχρονα για βελτιστοποίηση.

```

49 #4. Work on queries.csv
50 queries_df = pd.read_csv("queries.csv")
51 queries = queries_df["Text"].tolist()
52 query_ids = queries_df["ID"].tolist()
53
54 #a) Encode all queries at once to speed up the process
55 query_embeddings = model.encode(queries, show_progress_bar=True, batch_size=64)
56 faiss.normalize_L2(query_embeddings)
57
58 for k in [20, 30, 50]:
59
60     #b) One-time search for all queries
61     distances, indices = index.search(query_embeddings, k)
62
63     output_file = f"results2_{k}.txt"
64     with open(output_file, "w", encoding="utf-8") as f:
65         for i, qid in enumerate(query_ids):
66             #c) Save results in TREC format
67             for rank, (doc_idx, score) in enumerate(zip(indices[i], distances[i]), 1):
68                 doc_id = df.iloc[doc_idx]["ID"]
69                 f.write(f"{qid} Q0 {doc_id} {rank} {score} FAISS_TRANSFORMER\n")
70

```

Τα αποτελέσματα αποθηκεύτηκαν σε αρχεία TREC format ώστε να είναι συμβατά με το εργαλείο *trec_eval*. Έτσι, δημιουργήθηκαν 3 αρχεία: *results2_20*, *results2_30*, *results2_50* με την εξής μορφή:

```
Phase2 > ≡ results2_20.txt
1 Q01 Q0 193378 1 0.9309732913970947 FAISS_TRANSFORMER
2 Q01 Q0 210137 2 0.618607759475708 FAISS_TRANSFORMER
3 Q01 Q0 198900 3 0.6012340784072876 FAISS_TRANSFORMER
4 Q01 Q0 199915 4 0.6005395650863647 FAISS_TRANSFORMER
5 Q01 Q0 206230 5 0.5959801077842712 FAISS_TRANSFORMER
6 Q01 Q0 193836 6 0.5952179431915283 FAISS_TRANSFORMER
7 Q01 Q0 193373 7 0.5912319421768188 FAISS_TRANSFORMER
8 Q01 Q0 193715 8 0.5833245515823364 FAISS_TRANSFORMER
9 Q01 Q0 193375 9 0.5811314582824707 FAISS_TRANSFORMER
10 Q01 Q0 206011 10 0.5733577013015747 FAISS_TRANSFORMER
11 Q01 Q0 194890 11 0.569999098777771 FAISS_TRANSFORMER
12 Q01 Q0 206228 12 0.5632987022399902 FAISS_TRANSFORMER
13 Q01 Q0 204144 13 0.5615098476409912 FAISS_TRANSFORMER
14 Q01 Q0 210133 14 0.5613100528717041 FAISS_TRANSFORMER
15 Q01 Q0 206754 15 0.5582636594772339 FAISS_TRANSFORMER
16 Q01 Q0 212366 16 0.5548574924468994 FAISS_TRANSFORMER
17 Q01 Q0 211346 17 0.5546716451644897 FAISS_TRANSFORMER
18 Q01 Q0 212035 18 0.5535315871238708 FAISS_TRANSFORMER
19 Q01 Q0 193722 19 0.5502330660820007 FAISS_TRANSFORMER
20 Q01 Q0 206232 20 0.5497413873672485 FAISS_TRANSFORMER
```

Στιγμιότυπο ενός από τα αρχεία(*query_id Q0 doc_id rank score name*)

5. Στο τελευταίο βήμα χρησιμοποιήσαμε το εργαλείο αξιολόγησης *trec_eval* και τα μέτρα αξιολόγησης *MAP* (mean average precision) και *Precision@k* (ακρίβεια στα k πρώτα ανακτηθέντα κείμενα) για k = 5, 10, 15, 20 συγκρίνοντας τα αποτελέσματα μας με τις σωστές απαντήσεις του αρχείου *qrels.txt* παράγοντας τα αντίστοιχα αρχεία: *eval2_20*, *eval2_30*, *eval2_50* τα οποία περιέχουν το *map* και το *precision* για καθένα από τα δοθέντα k. Η εντολή που χρησιμοποιήθηκε για την δημιουργία αυτών των αρχείων είναι ουσιαστικά η: *trec_eval -m map -m P.5,10,15,20 qrels.txt results2_k.txt > eval2_k.txt* όπου k={20, 30, 50}.

```
Phase2 > ≡ eval2_20.txt
1 map all 0.3129
2 P_5 all 0.6600
3 P_10 all 0.5000
4 P_15 all 0.3933
5 P_20 all 0.3500
```

Το αρχείο *eval_20.txt* με τα αποτελέσματα του *trec_eval*

Πίνακας συνολικών αποτελεσμάτων του *trec_eval*:

	k=20	k=30	k=50
MAP	0.3121	0.3413	0.3744
P@5	0.6600	0.6600	0.6600
P@10	0.5000	0.5000	0.5000
P@15	0.3933	0.3933	0.3933
P@20	0.3500	0.3500	0.3500

Παρατηρούμε πως το map μεγαλώνει όσο μεγαλώνει και το πλήθος των εγγράφων και αυτό συμβαίνει λόγω του ότι εξετάζονται περισσότερα κείμενα και είναι αναμενόμενο. Το precision μένει σταθερό για κάθε k και αυτό επίσης είναι λογικό και δεν θα έπρεπε να αλλάζει εφόσον το $p < k$. Παρατηρούμε επίσης πως το precision μειώνεται όσο αυξάνεται το p και αυτό συμβαίνει γιατί τα κείμενα είναι ταξινομημένα κατά φθίνουσα σχετικότητα. Βρήκαμε, δηλαδή, τις ίδιες συσχετίσεις με την πρώτη φάση της εργασίας. Ωστόσο, υπενθυμίζουμε το αντίστοιχο πίνακάκι της φάσης 1:

	k=20	k=30	k=50
MAP	0.6895	0.7471	0.7701
P@5	0.8800	0.8800	0.8800
P@10	0.7300	0.7300	0.7300
P@15	0.7000	0.7000	0.7000
P@20	0.6300	0.6300	0.6300

Παρατηρούμε πως με την χρήση transformers έχουμε χειρότερα αποτελέσματα και αυτό γιατί με την Elasticsearch κάνουμε λεξική αντιστοίχιση (exact match), οπότε έχουμε υψηλό precision όταν υπάρχουν κοινές λέξεις ενώ με Transformers έχουμε σημασιολογική ομοιότητα όπου βρίσκει σημασιολογικά παρόμοια κείμενα αλλά μπορεί να χάνει exact matches. Στην ουσία αν το qrels.txt έχει μόνο exact matches, η Elasticsearch θα έχει καλύτερο MAP ενώ αν έχει και synonyms/semantic matches, οι transformers θα είχαν καλύτερο MAP. Οι transformers βρίσκουν διαφορετικού τύπου σχετικότητα, μπορεί να βρουν καινούργια, απροσδόκητα αποτελέσματα που το Elasticsearch θα είχε χάσει γιατί ψάχνει exact matches.

Τα αρχεία του φακέλου Phase2 είναι:

documents.csv (μας παρέχεται και περιλαμβάνει τα κείμενα)

queries.csv (μας παρέχεται και περιλαμβάνει τα ερωτήματα)

qrels.txt (μας παρέχεται και περιλαμβάνει τις σωστές απαντήσεις)

trec_eval.exe (το εργαλείο αξιολόγησης)

cyqwin1.dll (απαραίτητο αρχείο του εργαλείου)

[phase2.py](#) (το πρόγραμμα μας όπου μπορούμε να το τρέξουμε και να πάρουμε τα παρακάτω αρχεία)

[document_embeddings.npy](#) (το παράγει το πρόγραμμα και περιλαμβάνει τα επεξεργασμένα κείμενα σε μορφή διανυσμάτων(το δίνουμε εμείς έτοιμο για εξοικονόμηση χρόνου))

[results2_20](#) (το παράγει το πρόγραμμα και περιλαμβάνει τις απαντήσεις για τα 20 πρώτα ανακτηθέντα κείμενα)

[results2_30](#) (το παράγει το πρόγραμμα και περιλαμβάνει τις απαντήσεις για τα 30 πρώτα ανακτηθέντα κείμενα)

[results2_50](#) (το παράγει το πρόγραμμα και περιλαμβάνει τις απαντήσεις για τα 50 πρώτα ανακτηθέντα κείμενα)

[eval2_20](#) (το παράγει το πρόγραμμα και περιλαμβάνει τα αποτελέσματα της αξιολόγησης για τα 20 πρώτα ανακτηθέντα κείμενα)

[eval2_30](#) (το παράγει το πρόγραμμα και περιλαμβάνει τα αποτελέσματα της αξιολόγησης για τα 30 πρώτα ανακτηθέντα κείμενα)

[eval2_50](#) (το παράγει το πρόγραμμα και περιλαμβάνει τα αποτελέσματα της αξιολόγησης για τα 50 πρώτα ανακτηθέντα κείμενα)

Η εργασία πραγματοποιήθηκε κατά το χειμερινό εξάμηνο του έτους 2025-2026 στα πλαίσια του μαθήματος “Συστήματα Ανάκτησης Πληροφοριών” από τις φοιτήτριες του τμήματος Πληροφορικής στο Οικονομικό Πανεπιστήμιο Αθηνών, [Στυλιανή Μουμτζή \(3220127\)](#) και [Αναΐς Φαρχάτ \(3220280\)](#)..